Base de Données Avancées

COURS 3

Langage de Contrôle de Données et Gestion des utilisateurs avec MySQL

Plan de la présentation

- Gestion utilisateurs
  - Classification des utilisateurs
  - Création d'un utilisateur
  - Modification mot de passe
  - Renommer ou supprimer un user
- Gestion base des données
  - Création base des données
  - Sélection base des données
- Privilèges d'un user

Classification des utilisateurs

- Le **DBA** (*DataBase Administrator).* Il en existe au moins un. Une petite base peut n'avoir qu'un seul administrateur. Une base importante peut en regrouper plusieurs. Parmi les taches d'administration: – installation et mises à jour de la base et de ses outils, gestion de l'espace disque, gestion des utilisateurs..etc.
- **L'administrateur réseau** (peut être le DBA) se charge de la configuration des couches client pour les accès distants.
- **Les développeurs** qui conçoivent et mettent à jour la base, peuvent agir sur leurs objets (création et modification des tables, index, vues, etc.).
- **Les administrateurs d'application** qui gèrent les données manipulées par la ou les applications. Pour les petites et les moyennes bases, le DBA joue ce rôle.
- **Les utilisateurs** qui se connectent et interagissent avec la base.

Création d'un utilisateur

- Avoir le privilège `CREATE USER` ou `INSERT` sur la base système `mysql`. La syntaxe est la suivante :
- **`CREATE USER`** `utilisateur [IDENTIFIED BY [PASSWORD] '`*`motdePasse`*`'][,`*`utilisateur2`*` [IDENTIFIED BY[PASSWORD]'`*`motdePasse2`*`'];`
- `IDENTIFIED BY` *motdePasse* permet d'affecter un mot de passe (16 cars max, sensibles à la casse) à un utilisateur (16 cars max, sensibles à la casse).

`CREATE USER` `beille@localhost IDENTIFIED BY 'asc';`

- Les utilisateurs, une fois créés, ont le droits suivants
  - **Lecture écriture** sur la base `test`;
  - **Lecture seule** sur la base `information_schema`.

Liste des utilisateurs

- `(root@localhost)[mysql] mysql> SELECT User,Host FROM user;`
- L'utilisateur vide '' correspond à une connexion *anonyme*.
- La machine désignée par « *%* » indique que la connexion est autorisée à partir de tout site.
- La machine désignée par « *localhost* » spécifie que la connexion est autorisée en local.

User	Host
	%
	localhost
root	localhost
soutou	localhost

Modification mot de passe User

- Avoir le privilège `CREATE USER` (ou le privilège `UPDATE` sur la base de données `mysql`).
- Utilisez l'instruction `UPDATE` pour modifier le mot de passe d'un user, exemple:
- **`UPDATE`** `mysql.user`

  `SET Password =` **`PASSWORD`**`('salaam')`

  `WHERE User='beille' AND Host= 'localhost';`

  `FLUSH PRIVILEGES;`
- La fonction **`PASSWORD()`** est utilisée pour coder le password.
- L'instruction **`FLUSH PRIVILEGES`** recharge les tables systèmes.

Renommer ou supprimer un user

- Avoir le privilège `CREATE USER` (ou le privilège `UPDATE` sur la base de données `mysql`).
- **RENAME USER** utilisateur TO nouveauNom;
- **RENAME USER** beille@localhost **TO** souleh@localhost;
- **RENAME USER** souleh@localhost **TO** souleh@192.168.143.172;
- **RENAME USER** souleh@192.168.143.172 **TO** beille@localhost;
- **DROP USER** beille@localhost;

Gestion base des données

Création d'une base des données

- Avoir le privilège `CREATE` au niveau global pour créer des tables.
- **`CREATE {DATABASE | SCHEMA}`** `[IF NOT EXISTS]` *`nomBase`* `[ [DEFAULT] CHARACTER SET` *`nomJeu`* `] [ [DEFAULT] COLLATE nomCollation ];`
- `IF NOT EXISTS` évite une erreur dans le cas où la base de données existe déjà.
- `nomBase` désigne le nom de la base (64 caractères max).Les caractères « / », « \ », ou « . » sont exclus.
- `CHARACTER SET` indique le jeu de caractères associé aux données qui résideront dans les tables de la base.
- `COLLATE` définit la collation du jeu de caractères en question. La collation permet de définir la position des caractères dans le jeu. Par exemple, il sera possible de différencier « à » de « a » ou pas.

Exemple création de base

- **CREATE DATABASE** bdnew DEFAULT CHARACTER SET ascii COLLATE ascii_general_ci; (pour la langue anglaise)
- **CREATE DATABASE** bdnew2 DEFAULT CHARACTER SET gb2312; (pour les chinois).
- Le **jeu de caractères par défaut** est défini dans my.ini à l'aide de la variable default-character-set. donc possibilité de créer des bases de données associées à différents jeux de caractères au sein d'un même serveur.
- Possibilité de définir des jeux de caractères différents de celui de la base pour les tables, ou même pour chaque colonne :
- CREATE TABLE bdnew2.testChap3 (col CHAR(5) **CHARACTER SET** cp850 , col2 CHAR(4)) **CHARACTER SET** latin1;
- INSERT INTO bdnew2.testChap5 VALUES ('GTR','IUT');

Sélection d'une base des données

- L'instruction `USE` sélectionne une base de données qui devient active dans une session. **`USE`** *`nomBase;`*
- `CREATE TABLE bdnew.test (col3 CHAR(5), col4 CHAR(4)) CHARACTER SET latin1;`
- `INSERT INTO bdnew.test VALUES('ACTMP','IUT');`
- `USE bdnew2;`
- `SELECT col ,bdnew.test.col3 FROM testChap3, bdnew.test WHERE col2 = bdnew.test.col4;`

Modification/suppression d'une base

- Avoir le privilège `ALTER` sur la base de données pour la modifier. Syntaxe :
- **`ALTER {DATABASE`** *`nomBase`* `[ [DEFAULT] CHARACTER SET` *`nomJeu`* `] [ [DEFAULT] COLLATE` *`nomCollation`* `];`
- **`ALTER DATABASE`** `bdnew2  DEFAULT CHARACTER SET cp850;`
- Avoir le privilège `DROP` sur la base de données pour la supprimer. Syntaxe :
- **`DROP {DATABASE | SCHEMA}`** `[IF EXISTS]` *`nomBase`*`;`
- **`DROP DATABASE`** `bdnew2;`

Privilèges d’un utilisateur

Privilèges d'un user

- Un **privilège** est un droit d'exécuter une certaine instruction SQL (privilège système, exemple connexion), ou un droit relatif aux données des tables situées dans différentes bases (privilège objet). Il existe plusieurs niveaux de privilèges :
- **Global level** : s'appliquent à toutes les bases du serveur et sont stockés dans `mysql.user` (exemple: `GRANT CREATE ON *.*`).
- **Database level** : s'appliquent à tous les objets d'une base de données en particulier, sont stockés dans `mysql.db` et `mysql.host` (exemple: `GRANT SELECT ON bdtest.*)`.

Autres niveaux de privilèges

- **Table level** : s'appliquent à la globalité d'une table d'une base de données en particulier, sont stockés dans la table `mysql.tables_priv` (exemple : `GRANT INSERT ON bdtest.Avion`).
- **Column level** : s'appliquent à une des colonnes d'une table en particulier, sont stockés dans la table `mysql.columns_priv` (exemple : `GRANT UPDATE (nomComp)ON bdtest.Compagnie`).
- **Routine level** : privilèges globaux ou au niveau d'une base (`CREATE ROUTINE, ALTER ROUTINE, EXECUTE,` et `GRANT`) s'appliquant aux procédures cataloguées, sont stockés dans la table `mysql.procs_priv` (`mysql`) (exemple: `GRANT EXECUTE ON PROCEDURE bdtest.sp1`)

Table `mysql.user`

- *Privilèges objet (LMD) sur toutes les bases de données*:

```
SELECT Host, User, Select_priv,
Insert_priv, Update_priv, Delete_priv
FROM mysql.user;
```

Host	User	Select_priv	Insert_priv	Update_priv	Delete_priv
localhost	root	Y	Y	Y	Y
localhost		Y	Y	Y	Y
%		N	N	N	N
localhost	Paul	N	N	N	N

Table `mysql.user`

- *Privilèges objet (LDD) sur toutes les bases de données*:

```
SELECT Host, User,Create_priv,
Drop_priv,Index_priv, Alter_priv FROM
mysql.user;
```

Host	User	Create_priv	Drop_priv	Index_priv	Alter_priv
localhost	root	Y	Y	Y	Y
localhost		Y	Y	Y	Y
%		N	N	N	N
localhost	Paul	N	N	N	N

Table `mysql.user`

- *Privilèges systèmes (LCD) sur toutes les bases de données*:

  `SELECT Host,User,` **`Create_user_priv, Grant_priv,Show_db_priv`** `FROM mysql.user;`

Host	User	Create_user_priv	Grant_priv	Show_db_priv
localhost	root	Y	Y	Y
localhost		N	Y	Y
%		N	N	N
localhost	Paul	N	N	N

Table mysql.user

- *Privilèges à propos des vues sur toutes les bases de données :*

```
SELECT Host,User, Create_view_priv,
  Show_view_priv FROM mysql.user;
```

Host	User	Create_view_priv	Show_view_priv
localhost	root	Y	Y
localhost		N	N
%		N	N
localhost	Paul	N	N

Table `mysql.user`

- *Privilèges à propos des procédures cataloguées sur toutes les bases de données :*

```
SELECT Host,User, Create_routine_priv,
  Alter_routine_priv, Execute_priv FROM
  mysql.user;
```

Host	User	Create_routine_priv	Alter_routine_priv	Execute_priv
localhost	root	Y	Y	Y
localhost		N	N	Y
%		N	N	N
localhost	Paul	N	N	N

Table mysql.user

- *Privilèges à propos des restrictions d'utilisateur :*

```
SELECT Host, User, max_questions "Requetes", max_updates "Modifs" ,
       max_connections "Connexions", max_user_connections "Cx simult."
       FROM mysql.user;
+-----------+------+----------+--------+-----------+-----------+
| Host      | User | Requetes | Modifs | Connexions | Cx simult. |
+-----------+------+----------+--------+-----------+-----------+
| localhost | root |        0 |      0 |          0 |          0 |
| localhost |      |        0 |      0 |          0 |          0 |
| %         |      |        0 |      0 |          0 |          0 |
| localhost | Paul |        0 |      0 |          0 |          0 |
+-----------+------+----------+--------+-----------+-----------+
```

Attribution des privilèges (GRANT)

- L'instruction GRANT permet d'attribuer un (ou plusieurs) privilège(s) à propos d'un objet à un (ou plusieurs) bénéficiaire(s). L'utilisateur qui exécute cette commande doit avoir reçu lui-même le droit de transmettre ces privilèges (reçu avec la directive GRANT OPTION):

```
GRANT privilège [ (col1 [, col2...])] [,privilège2 ... ]
ON [{TABLE | FUNCTION | PROCEDURE}]
{nomTable | * | *.* | nomBase.*}
TO utilisateur [IDENTIFIED BY [PASSWORD] 'password']
[,utilisateur2 ...]
[ WITH [ GRANT OPTION ]
[ MAX_QUERIES_PER_HOUR nb ]
[ MAX_UPDATES_PER_HOUR nb2 ]
[ MAX_CONNECTIONS_PER_HOUR nb3 ]
[ MAX_USER_CONNECTIONS nb4 ] ];
```

Option de GRANT

- *privilège :* description du privilège (ex : SELECT, DELETE, etc.).
- *col* précise la ou les colonnes sur lesquelles se portent les privilèges SELECT, INSERT ou UPDATE (exemple : UPDATE(typeAvion) pour n'autoriser que la modification de la colonne typeAvion).
- GRANT OPTION : permet de donner le droit de retransmettre les privilèges reçus à une tierce personne.

Privilèges pour GRANT et REVOKE

privilege	Commentaire
ALL [PRIVILEGES]	Tous les privilèges.
ALTER	Modification de base/table.
ALTER ROUTINE	Modification de procédure.
CREATE	Création de base/table.
CREATE ROUTINE	Création de procédure.
CREATE USER	Création d'utilisateur.
CREATE VIEW	Création de vue.
DELETE	Suppression de données de table.
DROP	Suppression de base/table.
EXECUTE	Exécution de procédure.
INDEX	Création/Suppression d'index.
INSERT	Insertion de données de table.
SELECT	Extraction de données de table.
SHOW DATABASES	Lister les bases.
SHOW VIEW	Lister les vues d'une base.
SUPER	Gestion des déclencheurs.
UPDATE	Modification de données de table.
USAGE	Synonyme de « sans privilège », USAGE est utilisé pour conserver les privilèges précédemment définis tout en les restreignant avec des options.

Affectation des privilèges

`GRANT CREATE, DROP ON bdpaul.* TO 'Paul'@'localhost';`	Privilège système *database level* : `Paul` (en accès local) peut créer ou supprimer des tables dans la base `bdpaul`.
`GRANT INSERT, SELECT, DELETE, UPDATE(ISBN) ON bdpaul.Livre TO 'Paul'@'localhost';`	Privilège objet *table level* : `Paul` peut insérer, extraire, supprimer et modifier la colonne `ISBN` de la table `Livre` contenue dans la base `bdpaul`.
`GRANT ALTER ON bdpaul.Livre TO 'Paul'@'localhost';`	Privilège système *table level* : `Paul` peut modifier la structure ou les contraintes de la table `Livre` contenue dans la base `bdpaul`.
`GRANT SELECT(titre) ON bdpaul.Livre TO 'Jules'@'localhost' WITH GRANT OPTION;`	Privilège objet *column level* : `Jules` peut extraire seulement la colonne titre de la table `Livre` contenue dans la base `bdpaul`. Il pourra par la suite retransmettre éventuellement ce droit.
`GRANT CREATE ON *.* TO 'Jules'@'localhost';`	Privilège système *global level* : `Jules` peut créer des bases de données.
`GRANT USAGE ON bdpaul.* TO 'Jules'@'localhost' WITH MAX_QUERIES_PER_HOUR 50 MAX_UPDATES_PER_HOUR 20 MAX_CONNECTIONS_PER_HOUR 6 MAX_USER_CONNECTIONS 3;`	Privilège système *database level* : `Jules` ne peut lancer, chaque heure, que 50 `SELECT`, 20 `UPDATE`, se connecter 6 fois (dont 3 connexions simultanées) sur la base de données `bdpaul`.

Voir les privilèges (SHOW GRANTS FOR)

- La commande SHOW GRANTS FOR liste les différentes instructions GRANT équivalentes à toutes les prérogatives d'un utilisateur donné.
- **SHOW GRANTS FOR** *utilisateur;*

```
SHOW GRANTS FOR 'Jules'@'localhost';
+-----------------------------------------------------------------------------+
| Grants for Jules@localhost                                                  |
+-----------------------------------------------------------------------------+
| GRANT CREATE ON *.* TO 'Jules'@'localhost' IDENTIFIED BY PASSWORD
 '*6AE163FB9EE8BB011EB2E87316AA5BE563A6CDB7' WITH MAX_QUERIES_PER_HOUR 50
  MAX_UPDATES_PER_HOUR 20 MAX_CONNECTIONS_PER_HOUR 6 MAX_USER_CONNECTIONS 3 |
| GRANT SELECT (titre) ON `bdpaul`.`Livre` TO 'Jules'@'localhost'
       WITH GRANT OPTION                                                     |
+-----------------------------------------------------------------------------+
```

Voir les privilèges (SHOW GRANTS FOR)

```
SHOW GRANTS FOR 'Paul'@'localhost';
+-----------------------------------------------------------------------------+
| Grants for Paul@localhost                                                   |
+-----------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO 'Paul'@'localhost' IDENTIFIED BY PASSWORD
      '*6AE163FB9EE8BB011EB2E87316AA5BE563A6CDB7'                            |
| GRANT CREATE, DROP ON `bdpaul`.* TO 'Paul'@'localhost'                      |
| GRANT SELECT, INSERT, UPDATE (ISBN), DELETE, ALTER ON `bdpaul`.`Livre`
       TO 'Paul'@'localhost'                                                  |
+-----------------------------------------------------------------------------+
```

```
SHOW GRANTS FOR 'root'@'localhost';
+-----------------------------------------------------------------------------+
| Grants for root@localhost                                                   |
+-----------------------------------------------------------------------------+
| GRANT ALL PRIVILEGES ON *.* TO 'root'@'localhost' IDENTIFIED BY PASSWORD
      '*387E25FE2CF7ED941E43A76AD9402825401698FC' WITH GRANT OPTION           |
+-----------------------------------------------------------------------------+
```

Table `mysql.db`

- La table `mysql.db` décrit les prérogatives au niveau *database*. Ainsi la colonne `Db` indique la base de données.

```
SELECT Host, User, Db, Create_priv, Drop_priv, Alter_priv FROM mysql.
+-----------+------+---------+-------------+-----------+------------+
| Host      | User | Db      | Create_priv | Drop_priv | Alter_priv |
+-----------+------+---------+-------------+-----------+------------+
| %         |      | test\_% | Y           | Y         | Y          |
| %         |      | test    | Y           | Y         | Y          |
| localhost | Paul | bdpaul  | Y           | Y         | N          |
+-----------+------+---------+-------------+-----------+------------+
```

Table mysql.tables_priv

- La table mysql.tables_priv décrit les prérogatives objet au niveau *table.*

```
SELECT CONCAT(User,'@',Host) "Compte",
       CONCAT(Db,'.',Table_name) "Objet", Grantor, Table_priv
       FROM mysql.tables_priv;
+-----------------+--------------+----------------+---------------------------+
| Compte          | Objet        | Grantor        | Table_priv                |
+-----------------+--------------+----------------+---------------------------+
| Jules@localhost | bdpaul.Livre | root@localhost | Grant                     |
| Paul@localhost  | bdpaul.Livre | root@localhost | Select,Insert,Delete,Alter|
+-----------------+--------------+----------------+---------------------------+
```

Table `mysql.columns_priv`

- La table `mysql.columns_priv` décrit les prérogatives objet au niveau *column.*

```
SELECT CONCAT(User,'@',Host) "Compte", CONCAT(Db,'.',Table_name) "Objet",
       Column_name, Column_priv FROM mysql.columns_priv;
+-----------------+---------------+-------------+-------------+
| Compte          | Objet         | Column_name | Column_priv |
+-----------------+---------------+-------------+-------------+
| Jules@localhost | bdpaul.Livre | titre       | Select      |
| Paul@localhost  | bdpaul.Livre | ISBN        | Update      |
+-----------------+---------------+-------------+-------------+
```

Révocation de privilèges (REVOKE)

- La révocation d'un ou de plusieurs privilèges est réalisée par l'instruction `REVOKE`. Pour pouvoir révoquer un privilège, vous devez détenir au préalable ce même privilège avec l'option `WITH GRANT OPTION`.
- **REVOKE** *privilège* [(col1 [, col2...])] ON [{TABLE | FUNCTION | PROCEDURE}] {*nomTable* | * | *.* | *nomBase.**} FROM *utilisateur* ;
- **REVOKE ALL PRIVILEGES,** GRANT OPTION FROM *utilisateur* [, utilisateur2 ...] ; (avoir le privilège `CREATE USER` au niveau global ou le privilège `UPDATE` au niveau *database* sur la base `mysql`).

Révocation de privilèges (REVOKE)

Instruction faite par root	Explication

`REVOKE CREATE`
`ON bdpaul.*`
`FROM 'Paul'@'localhost';`
	Privilège système *database level* :
	`Paul` (en accès local) ne peut plus créer de tables dans la base `bdpaul`.

`REVOKE ALTER,INSERT,UPDATE(ISBN)`
`ON bdpaul.Livre`
`FROM 'Paul'@'localhost';`
	Privilège objet *table level* :
	`Paul` ne peut plus modifier la structure (ou les contraintes), insérer et modifier la colonne `ISBN` de la table `Livre` contenue dans la base `bdpaul`.

`GRANT USAGE ON bdpaul.*`
`TO 'Jules'@'localhost'`
`WITH`
`MAX_QUERIES_PER_HOUR 0`
`MAX_UPDATES_PER_HOUR 0;`
	Privilège système *database level* :
	`Jules` n'est plus limité en requêtes `SELECT` et `UPDATE` sur la base de données `bdpaul`. Ici c'est un `GRANT` qu'il faut faire, car il s'agit plus d'une restriction de connexion que d'une instruction SQL.

Accès distant vers MySQL

- Demander un accès au serveur de MySQL à partir d'une machine autre que celle contenant le serveur MySQL.
- MySQL doit être installé sur la machine cliente.
- Les deux machines (client/serveur) doivent être reliée par TCP/IP.
- Attribution des droits d'accès distant à l'utilisateur sur le serveur (192.168.4.8), exemple:
- **CREATE USER** `userd@192.168.4.3 IDENTIFIED BY 'sitedist';`
- **GRANT SELECT ON** `bdtest.tabtest` **TO** `'userd'@'192.168.4.3';`
- **mysql** `-h 192.168.4.8 -u userd -p` (commande de connexion depuis le client (192.168.4.3) vers le serveur).

Table mysql.host

- La table mysql.host est utilisée conjointement avec mysql.db et concerne les accès distants (plusieurs machines).
- Elle n'est employée que pour les droits au niveau *database,* indépendamment des utilisateurs.
- La structure est la même que celle de mysql.db, à l'exception de la colonne User qui n'est pas présente. Le couple de colonnes (Host, Db) est unique.

Caractère	Signification pour mysql.db		Signification pour mysql.host	
	colonne Host	colonne Db	colonne Host	colonne Db
%	toute machine	toute base	toute machine	toute base
' ' (chaîne vide)	consultez la table mysql.host	toute base	toute machine	toute base

Table mysql.host

- Pour les (INSERT, UPDATE, etc.), MySQL interroge la table user. Si accès non décrit, recherche dans les tables db et host.
- Si la colonne Host de la table db est renseignée en fonction de l'accès, l'utilisateur reçoit ses privilèges.
- Si la colonne Host de la table db n'est pas renseignée (' '), cela signifie que la table host énumère les machines qui sont autorisées à accéder à une base de données en particulier.
- La table mysql.host n'est mise à jour ni par GRANT, ni par REVOKE. Il faudra directement insérer (par INSERT), modifier (par UPDATE) ou supprimer (par DELETE) les lignes de cette table.

Host	Db	...
camparols.gtr.fr	%	(tous les privilèges à 'N')
%.gtr.fr	%	(tous les privilèges à 'Y')

Les vues en MySQL

- Depuis la version 5 de MySQL, la confidentialité des données est boosté par l'utilisation de vues *(views)* qui agissent comme des fenêtres sur la base de données.
- Les vues correspondent à ce qu'on appelle « le niveau externe » qui reflète la partie visible de la base de données pour chaque utilisateur.
- Elles masquent la structure des tables interrogées, ce qui renforce la confidentialité.
- Les utilisateurs ne devraient accéder aux données qu'en questionnant ces vues.
- Dans certains cas, la définition d'une vue temporaire est nécessaire pour écrire une requête qu'il ne serait pas possible de construire à partir des tables seules.

Création d'une vue en MySQL

- Avoir le privilège `CREATE VIEW` et les privilèges en `SELECT` des tables présentes dans la requête de définition de la vue.
- **`CREATE`** `[OR REPLACE] [ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]` **`VIEW`** `[`*`nomBase.`*`]`*`nomVue`* *`[(listecolonnes)]`* `AS` *`requêteSELECT`* `[WITH [CASCADED | LOCAL] CHECK OPTION];`
- `OR REPLACE` remplace la vue par la nouvelle définition, même si elle existait déjà (avoir le privilège `DELETE` sur la base).
- `ALGORITHM=MERGE` : la définition de la vue et sa requête sont fusionnées en interne (convient aux vues modifiables).
- `ALGORITHM=TEMPTABLE` : vue en **lecture seule**, utilisation d'une table temporaire (`TEMPORARY`) pour les résultats.

Création d'une vue en MySQL

- `ALGORITHM=UNDEFINED` (Aucune option) : MySQL choisit l'option à adopter (souvent `MERGE`) .
- `WITH CHECK OPTION` : garantit que toute mise à jour de la vue par `INSERT` ou `UPDATE` s'effectuera conformément au prédicat (condition) contenu dans la requête de définition.
- `LOCAL` et `CASCADED` (par défaut): déterminent la portée de la vérification du prédicat (condition) quand une vue est définie à partir d'une autre vue.
  - `LOCAL`: vérification restreint à la vue elle-même.
  - `CASCADED` : vérifications étendues aux autres vues source.
- La **requête de définition** ne peut interroger une table *temporaire*, ni contenir de paramètres ou de variables de session.

Catégories des vues

Il existe deux catégories des vues, en fonction de la nature de la requête de définition:

- **Vue simple** : vue **monotable**, sa requête de définition ne contient ni des *fonctions*, ni de *regroupement*, mais sa mis à jour est possible.
- **Vue complexe** : vue **multitables**, sa requête de définition contient des *fonctions*, un *regroupement*, et sa mis à jour n'est pas toujours possible.

Les vues monotables

- Une vue **monotable** est définie par une requête `SELECT` ne comportant qu'une seule table dans sa clause `FROM`.
- **`CREATE VIEW`** `PilotesAF` **`AS`** `SELECT * FROM Pilote WHERE compa = 'AF';`
- **`CREATE VIEW`** `Etat_civil` **`AS`** `SELECT nom, nbHVol, adresse, compa FROM Pilote;`
- Une fois créée, une vue s'interroge comme une table par tout utilisateur détenant le privilège en lecture (`GRANT SELECT ON nomVue TO`…). Exemple:
- `SELECT SUM(nbHVol) FROM` **`PilotesAF`**`;`
- `SELECT COUNT(*) FROM` **`Etat_civil`**`;`

Vues avec utilisation d'alias

- Les alias, s'ils sont utilisés, désignent le nom de chaque colonne de la vue.
- Leur utilisation sert à masquer les noms des colonnes de l'objet source.
- **CREATE OR REPLACE VIEW** PilotesPasAF (codepil,nomPil,heuresPil, adressePil, societe)AS SELECT * FROM Pilote WHERE NOT(compa = 'AF');
- **CREATE OR REPLACE VIEW** PilotesPasAF AS SELECT brevet *codepil, nom nomPil, nbHVol heuresPil, adresse adressePil,compa societe* FROM Pilote WHERE NOT(compa = 'AF');

Vue source d'une autre vue

- L'objet source d'une vue est en général une table, mais peut aussi être une vue. Il est possible d'utiliser des *alias* pour renommer les colonnes de la nouvelle vue.
- La vue suivante est définie à partir de la vue `PilotesPasAF` précédemment créée.
- **`CREATE OR REPLACE VIEW`** *`EtatCivilPilotesPasAF`* `AS SELECT nomPil, heuresPil, adressePil FROM PilotesPasAF;`

Vues en lecture seule

- L'option `ALGORITHM=TEMPTABLE` déclare la vue (lecture seule) non modifiable par `INSERT`, `UPDATE`, ou `DELETE`.
- **`CREATE OR REPLACE VIEW`** *`ALGORITHM=TEMPTABLE`* `PilotesPasAFLS AS SELECT * FROM Pilote WHERE NOT(compa='AF');`
- `INSERT INTO` **`PilotesPasAFLS`** `VALUES ('PL-8',FARAH',5,'ARTA','ASO');`
- **`ERROR 1288 (HY000):`** `The target table PilotesPasAFLS of the INSERT is not updatable.`
- `UPDATE` **`PilotesPasAFLS`** `SET nbHvol=nbHvol+2;`
- **`ERROR 1288 (HY000):`** `The target table PilotesPasAFLS of the UPDATE is not updatable.`

Vues modifiables

- L'option `ALGORITHM=MERGE` convient à la définition d'une vue modifiable (*updatable view*) par `INSERT`, `UPDATE`, ou `DELETE`.
- Ces modifications sont intrinsèque à cette vue (concerne que ses données).
- Pour mettre à jour une vue, il doit exister une correspondance réciproque entre les lignes de la vue et celles de l'objet source.
- Sa requête de définition doit respecter les critères suivants :
  - Pas de directive `DISTINCT`, de fonction (`AVG`, `COUNT`, `MAX`, MIN, `SUM`, ou `VARIANCE`),d'expression dans le `SELECT` ;
  - Pas de `GROUP BY`, `ORDER BY` ou `HAVING`.

Vues modifiables (exemples)

- INSERT INTO Etat_civil VALUES ('Radwan',10,‘Balbala','ASO') ;
- INSERT INTO Etat_civil VALUES ('Liban',20,‘Arta','ASO'); (impossible)
- DELETE FROM Etat_civil WHERE compa='ASO';
- UPDATE Etat_civil SET nbHVol=nbHVol*2 WHERE nom=‘Radwan';
- INSERT INTO PilotesAF VALUES ('PL-8','Farah',5, ‘Arta','AF');

Directive CHECK OPTION

- La directive WITH CHECK OPTION empêche un ajout ou une modification non conformes à la définition de la vue.
- **CREATE OR REPLACE VIEW** PilotesAF AS SELECT * FROM pilote WHERE compa = 'AF' **WITH CHECK OPTION;**
- INSERT INTO PilotesAF VALUES('PL-11','Teste',900,'Randa',**'AF'**);
- INSERT INTO PilotesAF VALUES ('PL-9','Caboche',600,'Rennes',**'ASO'**);
- UPDATE PilotesAF SET compa=**'ASO'**;
- ERROR **1369 (HY000):** CHECK OPTION failed 'bdtest.PilotesAF‘.

Vues complexes

- Une vue complexe contient, dans sa définition, plusieurs tables (jointures) ou une fonction appliquée à des regroupements ou à des expressions. Sa mise à jour n'est pas toujours possible.
- Pour qu'une vue complexe soit modifiable, les restrictions sont les suivantes :
  - La requête de définition ne doit pas contenir de sous-interrogation (jointure procédurale).
  - Il n'est pas possible d'utiliser d'opérateur ensembliste (sauf `UNION[ALL]`).

Vues complexes (exemples)

- **CREATE VIEW Pilotes_multi_AF** AS SELECT p.brevet, p.nom, p.nbHVol, c.ville, c.nomComp FROM Pilote p, Compagnie c WHERE p.compa=c.comp AND p.compa='**AF**';
- **CREATE VIEW Moyenne_Heures_Pil** AS SELECT compa, AVG(nbHVol) moyenne FROM Pilote GROUP BY compa;
- **INSERT INTO Moyenne_Heures_Pil** VALUES ('TAT',50);
- INSERT INTO Pilotes_multi_AF VALUES('PL-4', 'Test',400,‘Arta‘,’AD');ERROR **1394** Can not insert into join view 'bdtest.Pilotes_multi_AF' without fields list.

Mis à jour d'une vue multitable

- **UPDATE Pilotes_multi_AF** SET nbHVol=nbHVol*2;
- **UPDATE Pilotes_multi_AF** SET ville='**Orly**';
- DELETE FROM **Pilotes_multi_AF**; (impossible)
- *ERROR **1395** (HY000): Can not delete from join view 'bdtest.Pilotes_multi_AF'.*

```
SELECT brevet,nom,nbHVol
       FROM Pilotes_multi_AF;
+--------+-------------+---------+
| brevet | nom         | nbHVol  |
+--------+-------------+---------+
| PL-1   | Louise Ente |  900.00 |
| PL-2   | Paul Ente   | 1800.00 |
+--------+-------------+---------+
```

```
SELECT brevet,nom,ville
       FROM Pilotes_multi_AF;
+--------+-------------+-------+
| brevet | nom         | ville |
+--------+-------------+-------+
| PL-1   | Louise Ente | Orly  |
| PL-2   | Paul Ente   | Orly  |
+--------+-------------+-------+
```

Tables protégées par clé

- Une table est dite protégée par sa clé (*key preserved)* si sa clé primaire est préservée dans la clause de jointure et se retrouve en tant que colonne de la vue multitable (elle peut jouer le rôle de clé primaire de la vue).

```
CREATE VIEW
Vue_Multi_Comp_Pil
 AS SELECT c.comp,
     c.nomComp,
     p.brevet, p.nom,
     p.nbHVol
 FROM  Pilote p,
       Compagnie c
 WHERE p.compa=c.comp;
```

comp	nomComp	brevet	nom	nbHVol
AF	Air France	PL-1	Louise Ente	450.00
AF	Air France	PL-2	Paul Ente	900.00
SING	Singapore AL	PL-3	Paul Soutou	1000.00

Vue multitable modifiable (Critères)

- Une vue multitable modifiable (*updatable join view ou modifiable join view)* est une vue qui n'est pas définie avec l'option `ALGORITHM=TEMPTABLE` et dont sa requête de définition contient plusieurs tables dans la clause FROM.
- Aucune suppression n'est possible.
- Les insertions sont permises seulement en isolant toutes les colonnes d'une seule table source.

Vue multitable modifiable

- `DELETE FROM` **`Vue_Multi_Comp_Pil`** `WHERE comp='AF';` (impossible, erreur 1395!)
- `UPDATE` **`Vue_Multi_Comp_Pil`** `SET nbHVol = nbHVol * 3;` Query OK, 3 rows affected.
  Rows matched: 3 Changed: 3 Warnings: 0
- `UPDATE` **`Vue_Multi_Comp_Pil`** `SET nomComp =` **`'Dupond'`**`;` Query OK, 2 rows affected.
  Rows matched: 3 Changed: 2 Warnings: 0
- `INSERT INTO` **`Vue_Multi_Comp_Pil`** `(brevet,nom,nbHVol)VALUES('PL-5', 'Jama',250);`
- `INSERT INTO` **`Vue_Multi_Comp_Pil`** `(comp,nomComp) VALUES ('TAT', 'Test');`

Transmission des droits sur une vue

- si un utilisateur désire transmettre des droits sur une partie d'une de ses tables, il utilisera une vue. Seules les données appartenant à la vue seront accessibles aux bénéficiaires.
- Les privilèges objet qu'il est possible d'attribuer sur une vue sont les mêmes que ceux applicables sur les tables (`SELECT`, `INSERT`, `UPDATE` sur une ou plusieurs colonnes, `DELETE`).
- `GRANT` **`SELECT ON Vue_Multi_Comp_Pil`** `TO 'utest'@'localhost';`
- `GRANT` **`INSERT ON Vue_Multi_Comp_Pil`** `TO 'Jama'@'localhost';`

Modifier une vue (ALTER VIEW)

- Avoir les privilèges CREATE VIEW et DELETE au niveau d'une vue pour pouvoir la modifier.
- **ALTER** [ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}] **VIEW** [*nomBase.*]nomVue [(*listecolonnes*)] AS *requêteSELECT* [WITH [CASCADED | LOCAL] CHECK OPTION];
- Les transformations peuvent concerner toutes les parties d'une vue existante : la politique de création (ALGORITHM), la liste des colonnes, la requête, etc.
- **ALTER VIEW** ALGORITHM=TEMPTABLE **Vue_Multi_Comp_Pil** AS SELECT c.nomComp, p.brevet, p.nom FROM Pilote p, Compagnie c WHERE p.compa=c.comp;

Visualisation (SHOW CREATE VIEW)

- Pour pouvoir visualiser la requête de définition d'une vue, l'instruction que MySQL propose est la suivante :
- **SHOW CREATE VIEW** [*nomBase.]nomVue;*
- SHOW CREATE VIEW Vue_Multi_Comp_Pil;

Suppression vue (DROP VIEW)

- Avoir le privilège DROP sur une vue pour pouvoir la supprimer.
- **DROP VIEW [IF EXISTS]** [*nomBase.]nomVue [,nomBase2.]nomVue2 ;*
- **DROP VIEW IF EXISTS** PilotesPasAFLS;

FIN DU CHAPITRE